

GUÍA TÉCNICA DE REFERENCIA

UEFI, Legacy Boot y Sistemas de Formateo

Arquitectura de arranque · GPT/MBR · Sistemas de ficheros · Windows y Linux

Módulo	MF0490_3 — Gestión de Servicios en el Sistema Informático
Unidad	UD3 — Sistemas de Almacenamiento
Ámbito	UEFI · Legacy BIOS · MBR · GPT · Sistemas de ficheros · Formateo
Plataformas	Windows Server 2019 / Windows 10-11 · Ubuntu 22.04 LTS · Proxmox VE
Nivel	3 — Profesional · IFCT0109 Seguridad Informática

juanprofesor.com | Laboratorio Guacamole | IFCT0109

1. BIOS Legacy — El Sistema de Arranque Clásico

1.1 Qué es la BIOS

La BIOS (Basic Input/Output System) es el firmware que se ejecuta en el momento en que se enciende un ordenador, antes de que el sistema operativo tome el control. Fue diseñada en los años 70 por IBM y, a pesar de sus limitaciones, dominó la arquitectura de PC durante más de tres décadas. El firmware BIOS reside en un chip de memoria no volátil (originalmente ROM, luego EEPROM y Flash) en la placa base.

BIOS — Basic Input/Output System

Firmware de bajo nivel que inicializa el hardware del sistema (POST), detecta los dispositivos de arranque y transfiere el control al bootloader del sistema operativo. Opera en modo real de 16 bits, lo que limita el espacio de direccionamiento a 1 MB.

1.2 Proceso de arranque Legacy (BIOS + MBR)

El arranque en sistemas BIOS sigue una secuencia estrictamente definida conocida como 'boot sequence':

Fase	Nombre	Descripción
1	Power-On	La CPU se inicializa y apunta a la dirección F000:FFF0h donde reside el vector de reset de la BIOS.
2	POST	Power-On Self Test: la BIOS comprueba CPU, RAM, dispositivos de E/S y periféricos.
3	Hardware Init	Inicialización de controladores, detección de discos, configuración IRQs y DMA.
4	Boot Device Detection	La BIOS recorre la lista de dispositivos de arranque en el orden configurado.
5	MBR Load	Lee los primeros 512 bytes del disco (MBR) en la dirección 0x7C00 de RAM y transfiere control.
6	Bootloader	El código del MBR carga el bootloader de segunda etapa (ej: GRUB stage 2 o NTLDR/BOOTMGR).
7	OS Kernel	El bootloader carga el kernel del SO en memoria y cede el control.

1.3 MBR — Master Boot Record

El MBR es la estructura de datos que reside en los primeros 512 bytes del disco. Su estructura interna es:

Offset	Tamaño	Contenido
0x000	446 bytes	Código de arranque (bootloader de primera etapa)
0x1BE	64 bytes	Tabla de particiones: hasta 4 entradas de 16 bytes cada una
0x1FE	2 bytes	Firma de arranque: 0x55 0xAA (boot signature)

Limitaciones críticas del MBR

- Máximo 4 particiones primarias (se supera con partición extendida + lógicas).
- Tamaño máximo de disco: 2 TB (límite de direccionamiento LBA de 32 bits).
- Sin redundancia: si los 512 bytes del MBR se corrompen, el disco no arranca.
- Sin verificación de integridad: cualquier código en la dirección 0x7C00 se ejecuta sin comprobación.

1.4 Tabla de particiones MBR

Cada entrada de la tabla de particiones MBR ocupa 16 bytes con la siguiente estructura:

Bytes	Campo	Descripción
0	Estado	0x80 = activa (bootable), 0x00 = inactiva
1-3	CHS inicio	Cylinder-Head-Sector de inicio (obsoleto, ignorado con LBA)
4	Tipo	Identificador del sistema de ficheros (0x07=NTFS, 0x83=Linux ext, 0x82=swap...)
5-7	CHS fin	CHS de fin (obsoleto)
8-11	LBA inicio	Sector de inicio en LBA (32 bits)
12-15	Sectores	Número de sectores de la partición (32 bits → máx 2 TB)

```
# Ver tabla de particiones MBR en Linux
sudo fdisk -l /dev/sda

# Ver el MBR en hexadecimal (primeros 512 bytes)
sudo dd if=/dev/sda bs=512 count=1 | xxd | head -32

# Hacer backup del MBR (imprescindible antes de modificar)
sudo dd if=/dev/sda of=/backup/mbr_backup.bin bs=512 count=1

# Restaurar MBR desde backup (solo código, sin tabla de particiones)
sudo dd if=/backup/mbr_backup.bin of=/dev/sda bs=446 count=1
```

2. UEFI — Unified Extensible Firmware Interface

2.1 Qué es UEFI y por qué nació

UEFI (Unified Extensible Firmware Interface) es la especificación que reemplaza a la BIOS clásica. Fue desarrollada originalmente por Intel bajo el nombre EFI (Extensible Firmware Interface) para los sistemas Itanium a finales de los años 90, y posteriormente estandarizada por el UEFI Forum (consorcio formado por Intel, AMD, Microsoft, Apple, HP y otros fabricantes) en 2005. La especificación actual es UEFI 2.10 (2023).

¿Por qué se necesitaba UEFI?

La BIOS Legacy llegó a sus límites técnicos: no podía arrancar discos >2 TB, operaba en modo real de 16 bits (sin protección de memoria), no tenía interfaz gráfica ni red durante el preboot, y su proceso de arranque era inseguro (cualquier código en el MBR se ejecutaba sin verificación). UEFI resuelve todos estos problemas con una arquitectura modular y extensible.

2.2 Arquitectura UEFI

UEFI es fundamentalmente diferente a la BIOS: no es un programa de 16 bits monolítico, sino un entorno completo con su propio ejecutable PE32+ (el mismo formato que los .exe de Windows), protocolos de red, sistema de ficheros FAT32, shell de comandos y soporte para módulos cargables. Opera en modo protegido de 32 bits o modo largo de 64 bits.

Componente UEFI	Descripción	Equivalente BIOS
Firmware UEFI	Código principal del fabricante en Flash SPI	BIOS ROM
NVRAM	Variables persistentes (BootOrder, BootXXXX, SecureBoot...)	CMOS RAM
Boot Manager	Gestor de arranque integrado en el firmware	Boot sequence BIOS
UEFI Shell	Línea de comandos preboot (EFI Shell 2.0)	No existe
Driver UEFI	Módulos .efi cargables para hardware	INT 13h (ROM Option)
ESP	EFI System Partition: FAT32, contiene bootloaders	MBR + VBR
Secure Boot	Verificación criptográfica de bootloaders	No existe
Network Stack	PXE IPv4/IPv6 integrado	PXE parcial

2.3 EFI System Partition (ESP)

La EFI System Partition (ESP) es una partición especial de tipo FAT32 que UEFI usa como repositorio de bootloaders. A diferencia del MBR (512 bytes), la ESP puede contener múltiples bootloaders de distintos sistemas operativos, coexistiendo sin sobreescribirse.

EFI System Partition (ESP)

Partición FAT32 con tipo GUID C12A7328-F81F-11D2-BA4B-00A0C93EC93B (en GPT) o tipo 0xEF (en MBR). Tamaño recomendado: 100-550 MB. Montada típicamente en /boot/efi en Linux y accesible como EFI\... desde Windows. Contiene los ficheros .efi de cada bootloader.

Estructura típica de la ESP:

```
# Estructura de directorios de la ESP (montada en /boot/efi)
/boot/efi/
```

```

├── EFI/
│   ├── BOOT/
│   │   └── BOOTX64.EFI          # Bootloader por defecto (fallback)
│   ├── Microsoft/
│   │   ├── Boot/
│   │   │   ├── BCD              # Windows Boot Configuration Data
│   │   │   └── bootmgfw.efi     # Windows Boot Manager
│   │   └── Recovery/
│   ├── ubuntu/
│   │   ├── grubx64.efi         # GRUB2 para Ubuntu
│   │   ├── shimx64.efi        # Shim (Secure Boot)
│   │   └── grub.cfg
│   ├── proxmox/
│   │   └── grubx64.efi
│   └── NvVars                  # Variables NVRAM en sistemas sin flash NVRAM

```

2.4 Variables NVRAM y gestor de arranque UEFI

UEFI almacena la configuración de arranque en variables NVRAM persistentes. Las más importantes son:

Variable NVRAM	Descripción	Tipo
BootOrder	Lista ordenada de entradas de arranque (ej: 0001,0002,0003)	UINT16[]
BootXXXX	Entrada de arranque individual (path al .efi, descripción)	EFI_LOAD_OPTION
Boot Current	Entrada usada en el último arranque	UINT16
BootNext	Entrada a usar en el PRÓXIMO arranque (una sola vez)	UINT16
SecureBoot	1=Secure Boot activo, 0=desactivado	UINT8
SetupMode	1=modo configuración (sin Secure Boot), 0=modo usuario	UINT8
PK / KEK / db / dbx	Bases de datos de claves Secure Boot	Certificados X.509
Timeout	Segundos que el gestor UEFI espera antes de arrancar automáticamente	UINT16

```

# Ver variables NVRAM / entradas de arranque en Linux
sudo efibootmgr -v

# Crear entrada de arranque UEFI
sudo efibootmgr --create --disk /dev/sda --part 1 \
  --label 'Ubuntu 22.04' --loader '\EFI\ubuntu\shimx64.efi'

# Cambiar orden de arranque
sudo efibootmgr --bootorder 0001,0002,0003

# Establecer próximo arranque (una sola vez)
sudo efibootmgr --bootnext 0002

# Ver variables EFI disponibles
ls /sys/firmware/efi/efivars/

# Leer variable Secure Boot
cat /sys/firmware/efi/efivars/SecureBoot-8be4df61-93ca-11d2-aa0d-00e098032b8c | xxd

```

```
# Windows: gestión de entradas de arranque UEFI
# Ver entradas (desde CMD como Administrador)
bcdedit /enum firmware

# Copiar bootloader de Windows a ubicación de fallback
# (útil cuando UEFI no encuentra la entrada de Windows)
bcdedit /set {fwbootmgr} displayorder {bootmgr}

# Ver si Secure Boot está activo (PowerShell)
Confirm-SecureBootUEFI

# Ver estado detallado
msinfo32 # → Modo BIOS: UEFI / Estado de arranque seguro: Activado
```

2.5 Secure Boot

Secure Boot es una característica de UEFI que verifica la firma criptográfica de cada componente del proceso de arranque antes de ejecutarlo. Fue introducida con UEFI 2.2 y se convirtió en requisito para la certificación 'Windows 8 Compatible' de Microsoft.

Base de datos	Contenido	Función
PK (Platform Key)	Clave del fabricante OEM	Raíz de confianza. Controla KEK.
KEK (Key Exchange Key)	Claves de Microsoft + OEM	Controla db y dbx.
db (Signature Database)	Firmas/certificados de bootloaders permitidos	Lista blanca.
dbx (Forbidden Signatures)	Firmas revocadas / vulnerables	Lista negra.
MOK (Machine Owner Key)	Claves registradas por el usuario	Extensión para Linux

Secure Boot en Linux — Shim

La mayoría de distribuciones Linux (Ubuntu, Fedora, RHEL...) usan 'shim': un bootloader firmado por Microsoft que actúa como puente. Shim verifica GRUB, y GRUB verifica el kernel. Esto permite arrancar con Secure Boot sin desactivarlo. Para módulos del kernel de terceros (ej: drivers NVIDIA) se usa MOK Manager (mokutil).

```
# Verificar si Secure Boot está activo en Linux
mokutil --sb-state

# Enrollar clave MOK para módulo de kernel firmado
# 1. Generar clave de firma
openssl req -new -x509 -newkey rsa:2048 -keyout /root/mok.key \
  -out /root/mok.crt -days 3650 -subj '/CN=Mí Clave MOK/'

# 2. Importar en MOK Manager (pedirá contraseña y reinicio)
sudo mokutil --import /root/mok.crt

# 3. Firmar módulo del kernel
sudo /usr/src/linux-headers-$(uname -r)/scripts/sign-file \
  sha256 /root/mok.key /root/mok.crt /ruta/al/modulo.ko

# Verificar módulos firmados cargados
mokutil --list-enrolled
```

3. BIOS Legacy vs UEFI — Comparativa Completa

Característica	BIOS Legacy	UEFI
Año de origen	1975 (IBM PC)	2005 (especificación 1.0)
Modo de CPU	Real mode 16 bits	Protegido 32/64 bits
Tabla de particiones	MBR (obligatorio)	GPT (nativo) / MBR (CSM)
Límite de disco	2 TB (LBA 32 bits)	8 ZB (LBA 64 bits → prácticamente ilimitado)
Máx. particiones	4 primarias (+ lógicas)	128 particiones (especificación GPT)
Arranque múltiple	Sobreescritura del MBR	Entradas NVRAM independientes por SO
Interfaz gráfica	No (solo texto)	Sí (resolución completa, ratón, red)
Shell preboot	No	EFI Shell 2.0
Seguridad de arranque	Ninguna	Secure Boot (firmas criptográficas)
Red en preboot	PXE básico	PXE IPv4/IPv6 completo, HTTP Boot
Tiempo de arranque	~8-15 segundos	~2-5 segundos (inicialización paralela)
Drivers hardware	ROM Option (int 13h)	Módulos .efi cargables
Actualizaciones firmware	BIOS flash monolítico	Módulos independientes actualizables
Soporte 32 bits	Completo	Parcial (CSM requerido para OS 32-bit legado)
CSM (Compat. Support Module)	No aplica	Opcional: emula BIOS para SO legacy

3.1 Modo CSM (Compatibility Support Module)

El CSM es un componente opcional de UEFI que emula el comportamiento de la BIOS Legacy, permitiendo arrancar sistemas operativos y dispositivos que no son compatibles con UEFI. Cuando CSM está activo, el firmware expone la interfaz INT 10h, INT 13h y la tabla de particiones MBR, haciendo que el sistema se comporte como una BIOS clásica.

CSM: cuándo activar y cuándo no

ACTIVAR CSM: Si necesitas instalar Windows 7 de 32 bits, Linux antiguo (pre-2012), o arrancar desde dispositivos con firmware Option ROM antiguo. **DESACTIVAR CSM (recomendado):** En sistemas modernos con Windows 10/11 o Linux reciente en disco GPT. CSM desactivado es **OBLIGATORIO** para usar Secure Boot. Windows 11 requiere CSM desactivado y TPM 2.0.

3.2 Implicaciones en Windows

Escenario	BIOS+MBR	UEFI+GPT
Windows 10 instalación	Soportado (modo Legacy)	Soportado (modo nativo)
Windows 11 instalación	NO SOPORTADO (requiere UEFI+TPM 2.0)	Requerido
BitLocker	Funcional (sin TPM o con TPM 1.2)	Recomendado (TPM 2.0 + Secure Boot)

UEFI, Legacy Boot y Sistemas de Formateo · MF0490_3 UD3		IFCT0109 · juanprofesor.com
Partición de sistema	Partición activa MBR	ESP FAT32 + MSR (16-128 MB)
Gestor de arranque	BOOTMGR en partición activa	bootmgfw.efi en ESP
Configuración arranque	BCD (Boot Configuration Data)	BCD + entradas NVRAM EFI
Recuperación	WinRE en partición oculta	WinRE + Recovery Partition GPT
Rendimiento arranque	Estándar	Hasta 30-40% más rápido
Arranque rápido (Fast Boot)	No compatible	Compatible (S4 hibernation)

3.3 Implicaciones en Linux

Escenario	BIOS+MBR	UEFI+GPT
GRUB 2 instalación	grub-install /dev/sda (MBR)	grub-install --target=x86_64-efi
Partición de arranque	/boot ext4 (opcional)	ESP FAT32 montada en /boot/efi
Partición BIOS boot	No necesaria	BIOS boot partition (2MB) si GPT+BIOS
Secure Boot	No soportado	Shim + GRUB firmado (Ubuntu/Fedora)
Múltiples distribuciones	GRUB sobrescribe el anterior	Cada distro tiene su entrada NVRAM
Reparación arranque	grub-install + update-grub	efibootmgr + grub-install EFI
Kernel updates	update-grub automático	update-grub actualiza entrada EFI
ARM / nuevas arquitecturas	No compatible	Nativo (UEFI es estándar en ARM64)

4. Tablas de Particiones: MBR vs GPT

4.1 GPT — GUID Partition Table

GPT (GUID Partition Table) es el esquema de particionado que acompaña a UEFI, aunque también puede usarse con BIOS (con limitaciones). GPT supera todas las limitaciones de MBR mediante una estructura redundante y el uso de GUIDs (identificadores únicos globales de 128 bits) para identificar particiones.

Estructura GPT en disco

LBA 0: MBR protector (evita que herramientas antiguas destruyan el disco). LBA 1: Cabecera GPT primaria (CRC32 de verificación, localización de la tabla). LBA 2-33: Tabla de particiones primaria (128 entradas × 128 bytes = 16.384 bytes). LBA 34 → LBA -34: Datos de las particiones. LBA -33 → LBA -2: Tabla de particiones secundaria (backup). LBA -1: Cabecera GPT secundaria (backup).

Característica	MBR	GPT
Año de introducción	1983	2005 (con UEFI)
Tamaño máximo de disco	2 TB	8 ZB (zetabytes)
Número máximo de particiones	4 primarias (extendida ilimitadas)	128 por especificación
Identificación de partición	Tipo de 1 byte	GUID de 128 bits
Nombre de partición	No (solo tipo)	Nombre Unicode de 72 bytes
Redundancia	Sin backup	Tabla primaria + tabla secundaria (final del disco)
Integridad	Sin verificación	CRC32 de cabecera y tabla de particiones
MBR protector	No aplica	Sí: LBA 0 contiene MBR con partición tipo 0xEE
Soporte en Windows	XP/Vista/7/8/10/11 (arranque)	Vista+ (datos), 7+ (arranque con UEFI)
Soporte en Linux	Todos	Kernel ≥ 2.6.25 (2008)
Soporte en macOS	No (solo datos)	Nativo desde macOS 10.6

4.2 Tipos de partición GPT importantes

GUID del tipo	Nombre	Uso
C12A7328-F81F-11D2-BA4B-00A0C93EC93B	EFI System Partition	Contiene bootloaders UEFI (.efi)
E3C9E316-0B5C-4DB8-817D-F92DF00215AE	Microsoft Reserved (MSR)	Reservada por Windows (16-128 MB)
EBD0A0A2-B9E5-4433-87C0-68B6B72699C7	Basic Data	Partición de datos Windows (NTFS/FAT32)
DE94BBA4-06D1-4D40-A16A-BFD50179D6AC	Windows Recovery Environment	WinRE (recuperación de Windows)
0657FD6D-A4AB-43C4-84E5-0933C84B4F4F	Linux swap	Área de intercambio Linux
0FC63DAF-8483-4772-8E79-3D69D8477DE4	Linux filesystem	ext4, XFS, Btrfs, etc.

UEFI, Legacy Boot y Sistemas de Formateo · MF0490_3 UD3		IFCT0109 · juanprofesor.com
E6D6D379-F507-44C2-A23C-238F2A3DF928	Linux LVM	Physical Volume para LVM
A19D880F-05FC-4D3B-A006-743F0F84911E	Linux RAID	Componente de mdadm RAID
21686148-6449-6E6F-744E-656564454649	BIOS boot partition	2 MB para GRUB en disco GPT+BIOS

```
# Crear tabla GPT con gdisk (Linux)
sudo gdisk /dev/sdb
# o → p (print), n (new), t (type), w (write)

# Crear tabla GPT con parted
sudo parted /dev/sdb mklabel gpt
sudo parted /dev/sdb mkpart ESP fat32 1MiB 551MiB
sudo parted /dev/sdb set 1 esp on
sudo parted /dev/sdb mkpart primary ext4 551MiB 100%

# Ver tabla de particiones GPT
sudo gdisk -l /dev/sdb
sudo parted /dev/sdb print

# Convertir MBR a GPT (sin pérdida de datos – con gdisk)
# ATENCIÓN: hacer backup antes. Sólo seguro si no hay 4 particiones primarias.
sudo gdisk /dev/sda
# → w (write to GPT) – convierte automáticamente

# Windows: gestión de particiones GPT con diskpart
diskpart
DISKPART> list disk
DISKPART> select disk 1
DISKPART> convert gpt           # Convierte MBR a GPT (disco vacío)
DISKPART> create partition efi size=550
DISKPART> format fs=fat32 quick label='System'
DISKPART> assign letter=S
DISKPART> create partition msr size=128
DISKPART> create partition primary
DISKPART> format fs=ntfs quick label='Windows'
DISKPART> assign letter=C

# Verificar tabla de particiones
DISKPART> list partition

# PowerShell: ver esquema de disco
Get-Disk | Select-Object Number, Size, PartitionStyle
Get-Partition | Format-Table -AutoSize
```

5. Sistemas de Ficheros — Características y Elección

5.1 Comparativa de sistemas de ficheros

Sistema de ficheros	OS nativo	Max. volumen	Max. fichero	Journaling	Casos de uso principales
FAT32	Universal	8 TB	4 GB	No	ESP, USB, interoperabilidad
exFAT	Universal	128 PB	128 PB	No	Tarjetas SD, USB grandes
NTFS	Windows	256 TB	256 TB	Sí (metadata+data)	Windows OS, datos Windows
ReFS	Windows Server	35 PB	35 PB	No (CoW)	Storage Spaces, Hyper-V
ext4	Linux	1 EiB vol.	16 TB	Sí	Sistema raíz Linux, propósito general
XFS	Linux	8 EiB	8 EiB	Sí (metadata)	Ficheros grandes, almacenamiento logs
Btrfs	Linux	16 EiB	16 EiB	CoW	Snapshots, RAID software, subvolúmenes
ZFS	Linux/BSD	256 ZiB	16 EiB	CoW	Integridad máxima, pools de almacenamiento
HFS+	macOS	8 EiB	8 EiB	Sí	macOS legacy (reemplazado por APFS)
APFS	macOS	8 EiB	8 EiB	CoW	macOS 10.13+, SSD optimizado

5.2 NTFS — New Technology File System

NTFS es el sistema de ficheros nativo de Windows desde Windows NT 3.1 (1993). Utiliza una estructura basada en la Master File Table (MFT), donde cada fichero o directorio tiene al menos una entrada de 1 KB en la MFT. Las características avanzadas de NTFS lo convierten en el estándar para sistemas Windows de producción.

Característica NTFS	Descripción
MFT (Master File Table)	Base de datos de todos los ficheros y metadatos del volumen
Journaling	Log de transacciones en \$LogFile: protege contra corrupción ante caídas
USN Journal	Journal de cambios (\$UsnJrnl): registro de modificaciones para sync/backup
ACLs (permisos)	Control de acceso discrecional por usuario/grupo (herencia, propagación)
Cifrado (EFS)	Encrypting File System: cifrado transparente por usuario
Compresión	Compresión LZNT1 a nivel de fichero o directorio (transparente para apps)
Streams alternativos	Alternate Data Streams (ADS): datos ocultos asociados a un fichero
Sparse files	Ficheros dispersos: no ocupa espacio en disco para bloques de ceros

UEFI, Legacy Boot y Sistemas de Formateo · MF0490_3 UD3		IFCT0109 · juanprofesor.com
Hard links	Múltiples entradas MFT apuntando al mismo fichero	
Symlinks	Junctions y symbolic links (mklink)	
Cuotas	Límites de espacio por usuario	
Transaccional (TxF)	Transacciones ACID en operaciones de ficheros (Windows Vista+)	

5.3 ext4 — Fourth Extended Filesystem

ext4 es la evolución de ext3 y el sistema de ficheros más ampliamente utilizado en Linux. Introducido en el kernel 2.6.28 (2008), ext4 mantiene compatibilidad retroactiva con ext2/ext3 y añade soporte para volúmenes y ficheros mucho más grandes, además de mejoras de rendimiento significativas.

Característica ext4	Descripción
Extents	Bloques contiguos descritos por un único descriptor (vs. listas de bloques en ext2)
Journaling modes	journal (datos+metadata), ordered (default: solo metadata), writeback
Delayed allocation	Aplaza la asignación de bloques para optimizar la disposición en disco
Persistent preallocation	fallocate(): pre-reserva espacio para evitar fragmentación (multimedia)
Online defragmentation	e4defrag: desfragmentación sin desmontar el volumen
Timestamps nanosecond	Resolución de 1 nanosegundo en marcas de tiempo (vs 1 segundo en ext3)
Directory indexing	HTree (B-tree) para directorios con muchos ficheros
Large directory	Directorios con >32.000 subdirectorios
64-bit storage	Soporte de volúmenes hasta 1 EiB
Checksums	CRC32c en el journal y metadatos opcionales (extX_metadata_csum)

5.4 Btrfs — B-tree Filesystem

Btrfs (pronunciado 'butter fs' o 'b-tree fs') es un sistema de ficheros moderno para Linux basado en Copy-on-Write (CoW). Fue diseñado por Oracle Corporation e integrado en el kernel Linux 2.6.29 (2009). Su característica más destacada es el soporte nativo para snapshots, RAID software y subvolúmenes.

Copy-on-Write (CoW) — Concepto clave

En un sistema CoW, cuando se modifica un bloque de datos, el nuevo dato se escribe en un bloque libre (nunca sobrescribe el original). El puntero al bloque se actualiza una vez completada la escritura. Esto garantiza que el sistema de ficheros siempre quede en un estado consistente, sin necesidad de journaling tradicional.

Característica Btrfs	Descripción
Subvolúmenes	Unidades de almacenamiento independientes dentro del mismo filesystem
Snapshots	Instantáneas Copy-on-Write casi instantáneas (writable o read-only)
RAID integrado	RAID 0, 1, 10, 5, 6 a nivel de filesystem (sin mdadm)
Checksums	CRC32c para datos y metadatos: detecta corrupción silenciosa (bit rot)
Compresión	zlib, LZO, ZSTD por fichero o directorio
Deduplicación	Dedup fuera de línea (duperemove) y en línea (configuración avanzada)
Quotas	Límites de espacio por subvolumen con qgroups
Balance	Redistribución de datos entre dispositivos del pool
Scrub	Verificación de integridad de todos los bloques con corrección automática
Send/Receive	Serialización de snapshots para backup incremental eficiente

5.5 ZFS — Zettabyte File System

ZFS fue desarrollado por Sun Microsystems para Solaris (2005) y portado a Linux como OpenZFS. Es el sistema de ficheros más robusto disponible, diseñado específicamente para entornos de almacenamiento de alta disponibilidad. Combina sistema de ficheros y gestor de volúmenes en una única capa.

Característica ZFS	Descripción
zpool	Pool de almacenamiento que agrupa uno o más vdevs
vdev	Virtual device: disco simple, mirror, RAIDZ1/2/3, cache, log
Dataset	Subconjunto del pool: filesystem, volume, snapshot, clone
Checksums end-to-end	SHA-256 o SHA-512 en todos los bloques de datos y metadatos
Auto-reparación	Detecta y corrige errores automáticamente (con redundancia disponible)
ARC/L2ARC	Adaptive Replacement Cache en RAM + caché de segundo nivel en SSD
ZVOL	Volumen de bloque ZFS: emula un disco para VMs (usado en Proxmox)
Compresión	LZ4 (recomendado), GZIP, ZSTD — transparente y sin overhead de CPU
Deduplicación	Eliminación de bloques duplicados (requiere mucha RAM: ~5 GB/TB)
Encryption	Cifrado nativo AES-256 por dataset (ZFS native encryption)
Scrub	Verificación periódica de integridad de todo el pool

```
# Crear pool ZFS simple
sudo zpool create mi_pool /dev/sdb

# Pool mirror (RAID 1)
sudo zpool create mi_pool mirror /dev/sdb /dev/sdc
```

```
# Pool RAIDZ1 (RAID 5 equivalente, tolera 1 disco)
sudo zpool create mi_pool raidz /dev/sdb /dev/sdc /dev/sdd

# Crear dataset con compresión
sudo zfs create -o compression=lz4 mi_pool/datos

# Snapshot
sudo zfs snapshot mi_pool/datos@backup_2025

# Scrub de integridad
sudo zpool scrub mi_pool
sudo zpool status mi_pool
```

6. Tipos de Formateo — Conceptos y Diferencias

6.1 Formateo de alto nivel vs bajo nivel

Tipo de formateo	Qué hace	Tiempo	Datos recuperables	Cuándo usar
Formateo rápido (Quick)	Borra solo la tabla de ficheros (MFT o superbloque). Los datos permanecen.	Segundos	Sí (con herramientas de recuperación)	Reutilización interna, reinstalación del SO
Formateo completo (Full/Normal)	Borra tabla + verifica cada sector buscando sectores defectuosos. Sobrescribe con ceros.	Minutos-horas	Con esfuerzo (datos sobrescritos)	Preparación de disco para entrega o reutilización
Borrado seguro (Secure Erase)	Sobrescritura de todos los sectores con patrones aleatorios (DoD 5220.22-M, Gutmann, etc.).	Horas	Prácticamente imposible	Antes de dar de baja un sistema / datos confidenciales
Formateo de bajo nivel (LLF)	Redefinición de sectores físicos en el disco (solo fabricante en HDD modernos, SSD: ATA Secure Erase).	Horas	No (destrucción física de estructura)	Diagnóstico / preparación de fábrica
Sanitización criptográfica	Destruye la clave de cifrado del disco (SED / SSD con cifrado hardware).	Segundos	Imposible (datos cifrados sin clave)	SSD empresariales, NVMe con cifrado hardware

Nota sobre SSD y formateo

En unidades SSD el 'formateo completo' con escritura de ceros es INEFICIENTE y DAÑINO (aumenta desgaste del flash). Para borrado seguro en SSD usar: ATA Secure Erase (comando nativo del firmware del SSD) o NVMe Sanitize. En discos con cifrado hardware (Self-Encrypting Drive), destruir la clave AES equivale a destruir los datos instantáneamente.

6.2 Estándares de borrado seguro

Estándar	Organización	Pasadas	Método	Nivel de seguridad
DoD 5220.22-M	US Department of Defense	3	0x00 + 0xFF + aleatorio	Medio (obsoleto para HDD modernos)
DoD 5220.22-M (ECE)	US DoD (Extended)	7	Variantes del anterior + verificaciones	Alto
Gutmann	Peter Gutmann (1996)	35	Patrones específicos para técnicas de recuperación magnética	Muy alto (excesivo para HDD actuales)
NIST 800-88	NIST	1 (Clear) / 1+ (Purge)	Clear: sobrescritura lógica. Purge: ATA SE o destrucción física	Estándar de facto para sector público/empresarial
ATA Secure Erase	ATA/ATAPI	1 (Enhanced)	Comando firmware del disco (resetea celdas flash en SSD)	Alto para SSD / Medio para HDD
Crypto Erase	IEEE 2883	1	Destrucción de clave de cifrado (solo SED)	Máximo para discos con cifrado hardware

7. Formateo y Preparación de Discos en Linux

7.1 Herramientas de particionado

Herramienta	Tipo	Interfaz	Soporta	Uso recomendado
fdisk	CLI interactiva	Terminal	MBR (GPT limitado)	Particionado MBR legacy, discos <2 TB
gdisk	CLI interactiva	Terminal	GPT (MBR legacy)	Particionado GPT, conversión MBR→GPT
parted/partd	CLI + scripting	Terminal	MBR + GPT	Scripting, automatización, discos grandes
cfdisk	Pseudo-gráfica	ncurses TUI	MBR + GPT	Uso interactivo cómodo sin GUI
GParted	GUI completa	GTK/X11	MBR + GPT	Entornos gráficos, redimensionado visual
sgdisk	Scripting	Stdin/stdout	MBR + GPT	Automatización, backup de tablas de partición

7.2 Particionado completo para sistema UEFI+GPT en Linux

El siguiente procedimiento crea la estructura de particiones estándar para una instalación Linux con UEFI y GPT en un disco limpio (/dev/sdb como ejemplo):

```
# PASO 1: Verificar el disco y limpiar cualquier estructura previa
lsblk /dev/sdb
sudo wipefs -a /dev/sdb          # Borrar firmas de FS y tablas de partición
sudo sgdisk --zap-all /dev/sdb   # Borrar tabla GPT completa

# PASO 2: Crear tabla GPT y particiones con sgdisk
sudo sgdisk --clear /dev/sdb \
  --new=1:0:+550M --typecode=1:EF00 --change-name=1:'EFI System' \
  --new=2:0:+1G   --typecode=2:8300 --change-name=2:'Linux boot' \
  --new=3:0:+30G  --typecode=3:8200 --change-name=3:'Linux swap' \
  --new=4:0:0     --typecode=4:8300 --change-name=4:'Linux root'

# PASO 3: Verificar tabla creada
sudo sgdisk --print /dev/sdb
sudo parted /dev/sdb print

# PASO 4: Formatear cada partición
sudo mkfs.fat -F32 -n 'ESP' /dev/sdb1      # ESP → FAT32
sudo mkfs.ext4 -L 'boot' /dev/sdb2        # /boot → ext4
sudo mkswap -L 'swap' /dev/sdb3           # swap
sudo mkfs.ext4 -L 'root' /dev/sdb4        # / → ext4

# PASO 5: Montar y verificar
sudo mount /dev/sdb4 /mnt
sudo mkdir -p /mnt/boot /mnt/boot/efi
sudo mount /dev/sdb2 /mnt/boot
sudo mount /dev/sdb1 /mnt/boot/efi
sudo swapon /dev/sdb3
df -hT
```

7.3 Opciones avanzadas de mkfs

mkfs.ext4 — opciones importantes

```
# Crear ext4 con opciones de producción
sudo mkfs.ext4 \
  -L 'datos' \           # Etiqueta del volumen
  -b 4096 \              # Tamaño de bloque (4096 para uso general)
  -E lazy_itable_init=0 \ # Inicializar inodos completamente (más lento pero seguro)
  -E lazy_journal_init=0 \ # Inicializar journal completamente
  -m 1 \                 # Espacio reservado para root: 1% (defecto 5%)
  -j \                   # Activar journaling (ya por defecto en ext4)
  -J size=128 \          # Tamaño del journal en MB
  -O dir_index \         # Índices HTree para directorios
  -O extent \            # Extents (ya por defecto en ext4)
  -O has_journal \       # Confirmar journaling
  /dev/sdb4

# Ajustar parámetros después de formatear (tune2fs)
sudo tune2fs -i 0 -c 0 /dev/sdb4 # Sin límite de fsck por fecha/montajes
sudo tune2fs -m 2 /dev/sdb4      # Reducir espacio reservado a 2%

# Ver parámetros del sistema de ficheros
sudo tune2fs -l /dev/sdb4
sudo dumpe2fs /dev/sdb4 | head -40
```

mkfs.xfs — opciones importantes

```
# XFS para almacenamiento de ficheros grandes / logs
sudo mkfs.xfs \
  -L 'logs_xfs' \       # Etiqueta
  -b size=4096 \        # Tamaño de bloque
  -l size=128m \        # Tamaño del log (journal)
  -d agcount=8 \        # Número de allocation groups (paralelismo)
  /dev/sdb5

# XFS no puede reducirse — sólo expandirse:
sudo xfs_growfs /punto/montaje

# Verificar integridad XFS (disco desmontado)
sudo xfs_repair -n /dev/sdb5 # Solo verificar
sudo xfs_repair /dev/sdb5   # Reparar

# Info del filesystem XFS
sudo xfs_info /punto/montaje
```

Btrfs — creación y subvolumenes

```
# Crear Btrfs con compresión LZ0 por defecto
sudo mkfs.btrfs -L 'sistema' /dev/sdb4

# Btrfs en RAID 1 (dos discos, redundancia completa)
sudo mkfs.btrfs -d raid1 -m raid1 /dev/sdb /dev/sdc

# Crear subvolumenes (práctica recomendada para sistemas Linux)
sudo mount /dev/sdb4 /mnt
sudo btrfs subvolume create /mnt/@          # Subvolumen raíz
sudo btrfs subvolume create /mnt/@home      # Subvolumen /home
sudo btrfs subvolume create /mnt/@var       # Subvolumen /var
sudo btrfs subvolume create /mnt/@snapshots # Subvolumen para snapshots
sudo umount /mnt
```

```
# Montar con subvolúmenes
sudo mount -o subvol=@,compress=zstd /dev/sdb4 /mnt
sudo mount -o subvol=@home,compress=zstd /dev/sdb4 /mnt/home

# Crear snapshot de @ (writable)
sudo btrfs subvolume snapshot /mnt /mnt/@snapshots/snap_$(date +%Y%m%d)

# Listar subvolúmenes
sudo btrfs subvolume list /mnt
```

7.4 Borrado seguro en Linux

```
# MÉTODO 1: shred (sobrescritura con datos aleatorios)
# 3 pasadas aleatorias + 1 de ceros (DoD básico)
sudo shred -v -n 3 -z /dev/sdb

# MÉTODO 2: dd con /dev/urandom (lento pero completo)
sudo dd if=/dev/urandom of=/dev/sdb bs=4M status=progress

# MÉTODO 3: ATA Secure Erase (para HDD – RECOMENDADO)
# Verificar soporte
sudo hdparm -I /dev/sda | grep -i 'security'

# Poner el disco en modo frozen→sleep para des-freezarlo
sudo hdparm -y /dev/sda

# Establecer contraseña temporal (necesaria para el comando)
sudo hdparm --security-set-pass 'temporal' /dev/sda

# Ejecutar ATA Secure Erase
sudo hdparm --security-erase 'temporal' /dev/sda

# MÉTODO 4: NVMe Sanitize (para SSD NVMe – RECOMENDADO para SSD)
sudo nvme sanitize /dev/nvme0n1 -a 2 # 2=Block Erase (más común)
sudo nvme sanitize-log /dev/nvme0n1 # Ver estado del proceso

# MÉTODO 5: cryptsetup (cifrado + borrado de clave – quasi-instantáneo)
sudo cryptsetup open --type plain -d /dev/urandom /dev/sdb disco_efimero
sudo dd if=/dev/zero of=/dev/mapper/disco_efimero bs=4M status=progress
sudo cryptsetup close disco_efimero
```

7.5 Gestión de /etc/fstab — Montaje permanente

```
# Obtener UUID de cada partición (método recomendado sobre /dev/sdXY)
sudo blkid
lsblk -f

# Ejemplo de /etc/fstab completo para sistema UEFI+GPT+Btrfs
# <dispositivo>          <punto>   <fs>    <opciones>
# <dump> <pass>
UUID=xxxx-xxxx          /boot/efi vfat    umask=0077
0        2
UUID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx /         btrfs   subvol=@,compress=zstd
0        0
UUID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx /home     btrfs   subvol=@home,compress=zstd
0        0
```

```
UUID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx /var    btrfs    subvol=@var
0      0
UUID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx none     swap     sw
0      0

# Opciones de montaje importantes:
# noatime      - No actualizar atime en cada lectura (mejora rendimiento SSD)
# nodiratime   - No actualizar atime en directorios
# relatime     - Actualizar atime solo si es anterior a mtime (compromiso)
# errors=remount-ro - Remontar en solo lectura si hay error de ext4
# data=ordered - Journaling mode de ext4 (defecto)

# Verificar fstab sin reiniciar
sudo mount -a && echo 'fstab OK'

# Regenerar initramfs si se cambia fstab
sudo update-initramfs -u
```

8. Formateo y Preparación de Discos en Windows

8.1 Herramientas de gestión de discos

Herramienta	Interfaz	Disponibilidad	Capacidades principales
Administrador de discos (diskmgmt.msc)	GUI	Windows NT+	Particionar, formatear, asignar letras, cambiar estado básico/dinámico
DiskPart	CLI interactiva	Windows XP+	Toda la gestión de discos/particiones/volúmenes desde cmd
PowerShell (módulo Storage)	Scripting	Windows 8+	Automatización completa: Get-Disk, New-Partition, Format-Volume
Format.com	CLI	Windows NT+	Formateo simple de volúmenes existentes
Windows PE (WinPE)	Entorno preboot	Imagen ISO	Gestión de discos en fase de instalación/recuperación
fsutil	CLI	Windows XP+	Gestión avanzada de NTFS: cuotas, reparse points, sparse, USN journal
chkdsk	CLI	Windows NT+	Verificación y reparación de NTFS/FAT32/exFAT

8.2 Instalación limpia con DiskPart — Estructura UEFI+GPT

El siguiente procedimiento prepara un disco para una instalación limpia de Windows 10/11 con UEFI y GPT, siguiendo las recomendaciones de Microsoft para la estructura de particiones:

```
# Abrir CMD como Administrador (desde WinPE o instalación)
diskpart

# Seleccionar y limpiar el disco
DISKPART> list disk
DISKPART> select disk 0
DISKPART> clean                # Elimina MBR/GPT y todas las particiones
DISKPART> convert gpt          # Crear tabla GPT

# --- Estructura recomendada por Microsoft ---

# 1. Partición de recuperación de Windows (WinRE) – 499 MB
DISKPART> create partition primary size=499
DISKPART> format quick fs=ntfs label='Windows RE tools'
DISKPART> set id='de94bba4-06d1-4d40-a16a-bfd50179d6ac'
DISKPART> gpt attributes=0x8000000000000001 # Oculta + no automontable

# 2. EFI System Partition (ESP) – 100 MB (mínimo) / 550 MB (recomendado)
DISKPART> create partition efi size=550
DISKPART> format quick fs=fat32 label='System'

# 3. Microsoft Reserved Partition (MSR) – 128 MB
DISKPART> create partition msr size=128

# 4. Partición del sistema operativo Windows
DISKPART> create partition primary
DISKPART> format quick fs=ntfs label='Windows'
DISKPART> assign letter=C
```

```
# Verificar estructura resultante
DISKPART> list partition
DISKPART> list volume
DISKPART> exit
```

Estructura de particiones Windows 11 recomendada

Windows 11 requiere: 1) Partición WinRE (≥499 MB, NTFS, oculta). 2) ESP (≥100 MB, FAT32 — recomendado 550 MB). 3) MSR (128 MB, sin formato). 4) Partición del SO (≥64 GB, NTFS). Adicionalmente: TPM 2.0, Secure Boot habilitado, firmware UEFI. Sin estos requisitos, el instalador rechaza el disco.

8.3 PowerShell — Gestión avanzada de discos

```
# Inventario completo de discos y particiones
Get-Disk | Format-Table -AutoSize
Get-Partition | Select-Object DiskNumber, PartitionNumber, Size, Type, DriveLetter |
Format-Table
Get-Volume | Format-Table -AutoSize

# Inicializar disco nuevo y crear volumen completo
Initialize-Disk -Number 1 -PartitionStyle GPT
New-Partition -DiskNumber 1 -UseMaximumSize -DriveLetter D
Format-Volume -DriveLetter D -FileSystem NTFS -NewFileSystemLabel 'Datos' -
Confirm:$false

# Formateo NTFS con opciones avanzadas
Format-Volume -DriveLetter E `
  -FileSystem NTFS `
  -NewFileSystemLabel 'Servidor' `
  -AllocationUnitSize 65536 `      # 64 KB por cluster (ideal para SQL Server, VMs)
  -Full `                          # Formateo completo con verificación de sectores
  -Confirm:$false

# Crear partición específica GPT con tipo concreto
New-Partition -DiskNumber 1 -Size 550MB -GptType '{c12a7328-f81f-11d2-ba4b-
00a0c93ec93b}'

# Gestión de cuotas NTFS
Set-Acl 'D:\' ...
Enable-NtfsDiskQuota -DriveLetter D -WarningThreshold 800MB -WarningAction Report
```

8.4 Tamaño de unidad de asignación (Cluster Size)

El tamaño de cluster (Allocation Unit Size) en NTFS afecta al rendimiento y eficiencia del uso del espacio. Clusters más grandes mejoran el rendimiento con ficheros grandes pero desperdician espacio con ficheros pequeños (slack space).

Tamaño de cluster	Caso de uso óptimo	Tamaño MFT record	Observaciones
512 bytes	Volúmenes pequeños (<512 MB)	512 bytes	Máximo aprovechamiento del espacio
4 KB (defecto)	Propósito general, sistema operativo	1 KB	Equilibrio óptimo rendimiento/espacio
8 KB	Bases de datos OLTP moderadas	2 KB	Compatible con compresión NTFS
16 KB	Bases de datos medianas	4 KB	Sin compresión NTFS posible

UEFI, Legacy Boot y Sistemas de Formateo · MF0490_3 UD3			IFCT0109 · juanprofesor.com
32 KB	SQL Server, Exchange	4 KB	Recomendado por Microsoft para SQL
64 KB	Almacenamiento de VMs, backups	4 KB	Reducción del overhead de I/O
128-2048 KB	NAS/SAN de alto rendimiento	4 KB	Máximo throughput en lecturas secuenciales

8.5 Borrado seguro en Windows

```
# MÉTODO 1: Format con sobrescritura completa (lento pero seguro para HDD)
format D: /P:3      # 3 pasadas con datos aleatorios

# MÉTODO 2: cipher – borrado de espacio libre con ceros
cipher /w:C:\        # Sobrescribe el espacio libre de C: con 3 pasadas

# MÉTODO 3: SDelete (Sysinternals) – borrado de ficheros específicos
sdelete -p 3 -s C:\Datos\Confidencial\
sdelete -z C:\        # Sobrescribir espacio libre con ceros

# MÉTODO 4: diskpart clean all – sobrescritura completa del disco
diskpart
DISKPART> select disk 1
DISKPART> clean all   # Sobrescribe TODOS los sectores con ceros (lento: horas)

# MÉTODO 5: Eraser (GUI) – herramienta de borrado seguro gratuita
# Soporta DoD 5220.22-M, Gutmann, NIST 800-88, Schneier...
# https://eraser.heidi.ie/

# MÉTODO 6: ATA Secure Erase desde Windows (usando HDDease o Parted Magic)
# No hay comando nativo en Windows para ATA SE – usar herramienta de terceros
# o arrancar desde Linux para ejecutar hdparm --security-erase

# Verificar datos borrados (PowerShell)
# Intentar recuperar con Recuva / PhotoRec / TestDisk para verificar efectividad
```

8.6 chkdsk — Verificación y reparación NTFS

```
# Verificación básica (solo lectura, sin reparar)
chkdsk C:

# Reparación completa en el próximo reinicio (para la partición del sistema)
chkdsk C: /f /r
# /f = corregir errores del sistema de ficheros
# /r = localizar sectores defectuosos y recuperar datos legibles
# /x = forzar desmontaje del volumen antes de verificar
# /b = reevaluar clusters defectuosos (solo NTFS, muy lento)

# Verificación de otra unidad (sin reinicio necesario)
chkdsk D: /f

# Ver resultado del último chkdsk en el Event Log
Get-WinEvent -LogName System | Where-Object {$_.Id -eq 26212} |
  Select-Object -First 3 | Format-List TimeCreated, Message

# fsutil para estadísticas del sistema de ficheros
fsutil fsinfo ntfsinfo C:
fsutil volume diskfree C:
fsutil dirty query C:    # Comprobar si el volumen está 'sucio' (requiere chkdsk)
```


9. Reparación del Arranque — Casos Comunes

9.1 Reparar el arranque de Linux (GRUB2 + UEFI)

```
# Caso: GRUB no aparece después de instalar Windows (UEFI sobrescribe entrada)

# PASO 1: Arrancar desde USB Live (Ubuntu, Fedora...)

# PASO 2: Identificar particiones
lsblk -f
sudo fdisk -l

# PASO 3: Montar sistema existente en /mnt
sudo mount /dev/sda4 /mnt          # Partición raíz
sudo mount /dev/sda2 /mnt/boot     # /boot (si existe separada)
sudo mount /dev/sda1 /mnt/boot/efi # ESP

# PASO 4: Montar sistemas de ficheros virtuales
sudo mount --bind /dev /mnt/dev
sudo mount --bind /proc /mnt/proc
sudo mount --bind /sys /mnt/sys
sudo mount --bind /run /mnt/run

# PASO 5: Chroot al sistema instalado
sudo chroot /mnt

# PASO 6: Reinstalar GRUB UEFI
grub-install --target=x86_64-efi --efi-directory=/boot/efi \
  --bootloader-id=ubuntu --recheck

# PASO 7: Regenerar configuración de GRUB
update-grub

# PASO 8: Verificar entrada UEFI
efibootmgr -v

# PASO 9: Salir y reiniciar
exit
sudo umount -R /mnt
sudo reboot
```

9.2 Reparar el arranque de Windows (UEFI+GPT)

```
# Caso: Windows no arranca (entrada NVRAM perdida o BCD corrupto)

# PASO 1: Arrancar desde USB de instalación de Windows
# → Reparar el equipo → Solucionar problemas → Símbolo del sistema

# PASO 2: Identificar volúmenes
diskpart
DISKPART> list volume
# Identificar: volumen EFI (FAT32, ~550 MB) y volumen Windows (NTFS, grande)
DISKPART> select volume 1          # ESP
DISKPART> assign letter=S          # Asignar letra temporal a ESP
DISKPART> exit
```

```
# PASO 3: Reconstruir el BCD
bcdboot C:\Windows /s S: /f UEFI
# /s S: = ESP donde copiar los ficheros de arranque
# /f UEFI = tipo de firmware
# /l es-es = locale (opcional)

# PASO 4: Si BCD sigue corrupto – reconstruirlo completamente
bootrec /fixmbr          # Solo en modo Legacy (no aplicable en UEFI puro)
bootrec /fixboot
bootrec /scanos          # Buscar instalaciones de Windows
bootrec /rebuildbcd      # Reconstruir BCD desde cero

# PASO 5: Verificar entrada EFI creada
bcdedit /enum firmware

# PASO 6: Reparar con bootrec si hay error 0xc0000225
bcdedit /export C:\BCD_Backup
bcdedit /set {default} recoveryenabled Yes
reagentc /enable
```

9.3 Configurar arranque dual Windows + Linux (UEFI)

```
# Recomendación: instalar Windows PRIMERO, luego Linux
# Windows instala su bootloader en la ESP y lo marca como primario en NVRAM
# Linux (Ubuntu/Fedora) detecta Windows y lo añade automáticamente a GRUB

# Verificar que ambos SO están en la ESP:
ls /boot/efi/EFI/
# Debería mostrar: Boot/ Microsoft/ ubuntu/ (u otra distro)

# Si Windows no aparece en GRUB, ejecutar:
sudo update-grub
# GRUB 2 ejecuta os-prober que detecta Windows automáticamente

# Si os-prober está desactivado (Ubuntu 22.04+):
sudo nano /etc/default/grub
# Añadir/descomentar: GRUB_DISABLE_OS_PROBER=false
sudo update-grub

# Ver entradas UEFI (ambos SO deben aparecer)
sudo efibootmgr -v

# Cambiar orden: poner Windows primero en NVRAM
sudo efibootmgr --bootorder 0001,0002 # 0001=Windows, 0002=Linux

# Para que Windows no sobrescriba la entrada de GRUB:
# → Ir a Configuración UEFI → Secure Boot → Boot Override
# → Marcar GRUB/Ubuntu como primera opción en el firmware
```

10. Guía de Decisión — Qué usar en cada caso

10.1 ¿UEFI o Legacy BIOS?

Situación	Recomendación	Motivo
Hardware moderno (2012+)	UEFI + GPT	Mejor rendimiento, seguridad y límites de disco
Windows 11	UEFI obligatorio	Requisito del sistema: UEFI + Secure Boot + TPM 2.0
Linux moderno	UEFI + GPT (recomendado)	Soporte Secure Boot, arranque múltiple más fácil
Windows XP/Vista/7 (32-bit)	Legacy BIOS + MBR	No tienen drivers UEFI nativo 32-bit sin CSM
Linux antiguo (pre-2010)	Legacy BIOS + MBR	Sin soporte UEFI en kernels antiguos
Servidor con discos >2 TB	UEFI + GPT obligatorio	MBR no puede direccionar más de 2 TB
Arranque dual Win+Linux	UEFI + GPT	Cada SO tiene su propia entrada NVRAM sin conflicto
PXE boot empresarial	UEFI	PXE IPv4/IPv6 nativo, HTTP Boot, HTTPS Boot

10.2 ¿Qué sistema de ficheros elegir?

Caso de uso	Windows	Linux	Motivo
Partición del SO	NTFS	ext4 o Btrfs	Nativos y optimizados para cada SO
Disco de datos compartido (Win+Linux)	exFAT	exFAT	Soportado por ambos sin drivers adicionales
Servidor de ficheros Windows	NTFS / ReFS	—	ReFS para Storage Spaces, NTFS para compatibilidad
Almacenamiento de VMs	NTFS (64 KB cluster)	ext4 / XFS / ZFS	Clústers grandes reducen overhead de I/O
Backup / snapshots	ReFS	Btrfs / ZFS	Copy-on-Write nativo, snapshots eficientes
Logs del sistema / SIEM	NTFS	XFS	XFS optimizado para escritura de ficheros grandes
NAS / almacenamiento masivo	ReFS / Storage Spaces	ZFS / Btrfs RAID	Integridad de datos, escalabilidad
USB / tarjeta SD (portabilidad)	exFAT	exFAT	Universal, sin límite de 4 GB de FAT32
Servidor de base de datos	NTFS (32-64 KB)	XFS / ext4	Clusters grandes para SQL, ext4 para PostgreSQL
Disco de swap	No aplica	swap (Linux dedicado)	Área de intercambio del kernel

10.3 ¿Qué tipo de formateo usar?

Situación	Tipo de formateo	Herramienta
Reinstalación del SO (disco propio)	Formateo rápido	Windows installer / mkfs.*
Disco nuevo para servidor	Formateo completo + verificación de sectores	format /P:1 / mkfs -c

UEFI, Legacy Boot y Sistemas de Formateo · MF0490_3 UD3		IFCT0109 · juanprofesor.com
Disco de VM en Proxmox	Formateo rápido (el hipervisor gestiona la integridad)	mkfs.ext4 / mkfs.xfs
SSD para reutilización interna	ATA/NVMe Secure Erase	hdparm / nvme-cli / fabricante
HDD entregado a terceros	Borrado DoD 5220.22-M (3 pasadas) o NIST 800-88	shred / SDelete / Eraser
Datos clasificados / confidenciales	Degaussing + destrucción física	Servicio certificado
SSD con cifrado hardware (SED)	Crypto Erase (destruir clave AES)	Herramienta del fabricante
Disco de producción sospechoso de compromiso	Borrado seguro + nuevo SO en disco limpio	shred + reinstalación total

